

Objektumok életrajza, létrehozás, inicializálás, másolás, megszüntetés. Dinamikus, lokális és statikus objektumok létrehozása. A statikus adattagok és metódusok, valamint szerepük a programozásban. Operáció és operátor overloading a JAVA és C++ nyelvekben.

Kivételkezelés

OBJEKTUMOK ÉLETRAJZA, LÉTREHOZÁS, INICIALIZÁLÁS, MÁSOLÁS, MEGSZÜNTETÉS

- OOP esetén az objektum egy entitás, mely állapottal (attribútumok), viselkedéssel (metódusok) és egyedi identitással rendelkezik

Objektumok létrehozása

- objektumok létrehozása kötelezően a **new** operátorral történik
 - lefoglalja a memóriát a heap-en, majd meghívja a konstruktort és visszaad egy referenciát (JAVA) vagy pointert (C++) az objektumra
- **JAVA:** MyClass obj = new MyClass();
- **C++**
 - itt is **new operátorral** történik az objektum létrehozása, de csak opcionális
 - MyClass obj;
 - automatikusan létrejön a verembe (stack) az objektum
 - MyClass *obj = new MyClass();
 - dinamikus a heap-en jön létre az objektum, majd a konstruktor kerül meghívásra és egy pointert ad vissza az objektumra

Objektumok inicializálása

- **konstruktor** segítségével történik az objektumok inicializálása
 - speciális tagfüggvény, metódus, amely az objektum példányosításakor (létrehozáskor) fut le
 - feladata az adattagok kezdeti értékének beállítása (**inicializálás**)
 - a konstruktor tagfüggvény neve megegyezik az osztály nevével és nincs visszatérési típusa
 - **default konstruktor**
 - paraméter nélküli konstruktor
 - ha nem írunk sajátot, a fordító generál egyet
 - **paraméteres konstruktor**
 - lehetővé teszi az objektum állapotának tesztelését a létrehozás pillanatában
 - **másoló konstruktor (copy constructor)**
 - paramétere egy már létező objektum az adott osztályból és erről készít másolatot

- inicializálási különbségek
 - JAVA esetén az adattagok automatikusan alapértelmezett értéket kapnak, ha nem inicializáljuk őket
 - számok: 0, objektum referenciák: null
 - C++ esetén a primitív típusok (pl: int) értéke definiálatlan marad, ha nem adjuk meg explicit, ez memóriahibákhoz vezethet
 - C++-ban javasolt az **inicializáló lista** használata a konstruktor törzse előtt mert ez hatékonyabb
 - `Kurzus(string nev, string kod) : nev(nev), kod(kod), max(25) {}`

Objektumok másolása (Shallow vs Deep copy)

- amikor egy objektumot egy másiknak értékül adunk, nem mindegy mi másolódik le
- **Sekély másolás (Shallow copy)**
 - csak az objektum közvetlen adattagjai (állapot) másolódik le
 - ha az objektum pointert (C++) vagy referenciát (JAVA) tartalmaz egy másik memóriaterületre akkor mindkét objektum ugyanarra a területre fog mutatni
 - ez egy probléma, mert ha az egyik objektum módosítja vagy törli a közös adatot, a másik objektum állapota is megváltozik vagy érvénytelenné válik
- **Mély másolás (Deep copy)**
 - az objektum által mutatott (dinamikusan lefoglalt) adatokat le kell másolni egy új memória területre is, így a két objektum teljesen független lesz
- JAVA esetén a sajátosságok:
 - az = operátor csak referenciát másol (se nem sekély, se nem mély másolás)
 - mindkét változó ugyanarra az objektumra mutat, tehát maga az objektum nem másolódik
 - valódi másoláshoz a clone() metódus vagy a másoló konstruktor szükséges
 - `Object.clone()` csak sekély másolást végez
- C++ esetén a sajátosságok:
 - itt van másoló konstruktor (`Kutya(const Kutya& masik)`) és értékadó operátor (`operator=`) avagy másoló operátor
- ha az osztályunk dinamikus memóriát kezel, nekünk kell megírni ezeket a mély másoláshoz (ez a hármas szabály része)

Objektumok megszüntetése (destrukció)

- az objektum életciklusának végén a lefoglalt erőforrásokat, mint a memória vagy a fájlok fel kell szabadítani
- **C++ (determinisztikus)**
 - az objektum megszűnésekor automatikusan lefut a destruktor: `~Kutya()`
 - determinisztikus mert pontosan tudjuk mikor fut le:
 - lokális objektum esetén a blokk végén
 - dinamikus objektum esetén a delete hívásakor

- lehetővé teszi a **RAII** (Resource Acquisition Is Initialization) elvét:
 - az erőforrást a konstruktorba foglaljuk le és a destruktorba garantáltan felszabadítjuk
- C++-ban ha elfelejtjük megírni a destruktor egy dinamikus memóriát használó osztálynál akkor memóriaszivárgás (memory leak) lép fel, mert az objektum törlésekor a belső pointer által mutatott terület nem szabadul fel
- a destruktorban először az adattagok destruktorai futnak le (már aminek van, pl. int-nek nincs, de Stringnek vagy saját osztályoknak van) ellentétes sorrendbe mint ahogy deklaráltuk az adattagokat (tehát letről fel)
- **JAVA (nem-determinisztikus)**
 - nincs destruktor
 - **Garbage Collector (szemétgyűjtő)**
 - ez automatikusan figyeli a memóriát a háttérben és felszabadítja azokat az objektumokat, amikre már nincs referencia
 - hátránya, hogy nem tudjuk pontosan mikor fut le, ezért a nem-memória jellegű erőforrásokat (pl: nyitott fájl) kézzel kell lezárni (try-with-resources vagy close())

OBJEKTUMOK LÉTREHOZÁSÁNAK MÓDJAI ÉS A MEMÓRIA KEZELÉSE

Memóriák felosztása

- a program futása során a memória (RAM) több logikai részre oszlik, melyek meghatározzák az odakerülő adatok élettartalmát és elérési sebességét
- **Stack (verem)**
 - gyors és automatikus elérésű memória, de korlátozott
 - itt tárolódnak a lokális változók és a függvényhívások adatai (paraméterek, visszatérési címek)
 - ezek az adott blokk végén felszabadításra kerülnek
- **Heap (kupac)**
 - általános célú, dinamikus memória
 - itt jönnek létre a futás közben igényelt objektumok
 - elérése és az objektum alkotása lassabb mint a veremé
 - mert heap-en meg kell keresni és lefoglalni a szabad memóriaterületet, míg stack-en csak a stack pointert kell eltolni
- **Static / Constant storage vagy Adatszegmens**
 - itt maga a programkód tárolódik, illetve a konstans értékek és statikus (osztálysztintű) változók
 - egy adott blokk lefutása után is megmaradnak az osztálysztintű változók

Lokális (automatikus) objektumok

- egy függvényen vagy blokken ({}) belül jönnek létre, élettartalmuk a blokk végig tart
 - automatikusan létrejönnek és megszűnnek a metódushívások során
- **stack**-en tárolódnak
- **C++**
 - létrehozásuk egyszerű deklarációval történik: `Kutya k;`
 - a blokk végén a destruktoruk garantáltan azonnal lefut
- **JAVA**
 - nincs a JAVA-ban valódi lokális objektum, mivel minden objektum a Heap-ben tárolódik, csak az objektumra mutató referencia (maga a változó) lokális és stack-alapú

Dinamikus objektumok

- élettartamuk nem kötődik egy adott blokkhoz, a programozó (vagy a futtató környezet) vezérli őket és futásidőben jönnek létre
- **heap**-en tárolódnak
 - lassabb az objektum alkotás itt mint stack-en
- mindkét nyelv esetén explicit a *new* operátorral történik, ez lefoglalja a memóriát és meghívja a konstruktort, de a felszabadításban vannak különbségek:
 - **C++** esetén:
 - a programozó felelőssége a felszabadítás, azaz minden *new*-hoz kell egy *delete* vagy *delete[]*, különben memóriaszivárgás (memory leak) lép fel
 - **JAVA** esetén:
 - **Garbage Collector** kezeli a dinamikus objektumok felszabadítását
 - ha egy objektumra már egyetlen élő referencia sem mutat, akkor a GC valamikor felszabadítja azt a memóriaterületet

Statikus (globális) objektumok

- a statikus objektumok élettartalma a program teljes futása alatt fennáll
 - első használat (vagy programindulás) előtt jönnek létre és a program megállásakor szűnnek meg
- példányosítás nélkül elérhetők
- a *static* kulcsszóval deklarált objektumok az adatszegmensben (Static Storage) kapnak helyet mindkettő nyelv esetén
 - az osztály betöltésekor jönnek létre és a program végéig tart az élettartalmuk

STATIKUS ADATTAGOK ÉS METÓDUSOK

- a static kulcsszó lényege, hogy az ezzel ellátott adattagok és metódusok az osztályhoz tartoznak és nem külön a példányokhoz
 - egy osztály adattagjai alapesetben minden egyes példányban külön-külön léteznek (pl: minden Kutya objektumnak van egy saját név mezője)
 - a statikus adattagok és metódusok az osztály példányosítása nélkül elérhetők
 - mivel nem használnak példányszintű változókat
- **2 típus objektumokra**
 - **Példányszintű (instance)**
 - ahány objektum, annyi példány van belőle a memóriában
 - **Osztálysintű (static)**
 - egész program futása alatt egyetlen egy közös példány létezik belőle, amit az osztály összes példánya, objektuma megoszt egymással

Statikus adattagok

- ezen az adattagok az osztályhoz tartoznak, minden példány osztozik rajtuk
 - pl: egy számláló, mely követi, hogy az adott osztályból mennyi példány került létrehozásra
 - memóriába csak egy példány van a statikus adattagokból
- nem szükséges példányosítani az osztályt az elérésükhöz (pl: Kutya.statikusValtozo)
 - fontos, hogy a láthatósági módosítók ezekre is érvényesek, private esetén kell egy publikos getter/setter metódus pár kell (persze ez is statikus metódus lesz)
- **JAVA** esetén:
 - az osztályon belül deklaráljuk és inicializálhatjuk is akár
 - az osztály betöltésekor jönnek létre a statikus adattagok
- **C++** esetén:
 - az osztályon belül csak deklaráljuk, de az osztályon kívül (a .cpp fájlban) kell definiálni és helyet foglalni neki
 - C++17 óta a static inline kulcsszóval az adattag definiálásakor is megoldható a definiálás
 - ```
class Kutya{
 static int db1;
 static inline int db = 10; //C++17 óta jo
 static const int max_db = 100; //mivel ott a const ezért így jo
 static int xd = 0; //HIBA
}
int Kutya::db1 = 0;
```
    - ha egy adattagot csak az osztályban deklarálunk, de kívül nem definiáljuk akkor a linker (szerkesztő) hibát fog jelezni (undefined reference), mert nem talált memória címet a változóhoz

## Statikus metódusok

- ezen metódusok az osztályhoz tartoznak és csak statikus adattagokat és más statikus metódusokat érhetnek el
  - statikus metódusokban nincs **this** referencia, mivel nem egy konkrét objektumon fut
  - ha nem statikus adattagokat vagy metódusokat akarunk elérni akkor a statikus metódusba példányosítanunk kell a kívánt osztályból egy objektumot
  - statikus metódus nem lehet virtuális
- szerepük:
  - **segédfüggvények (utility)**
    - nincs szükségük belső állapotra, csak a bemenő paraméterekre
    - Math.sqrt() JAVA-ban vagy std::max() C++-ban
  - **factory metódusok**
    - objektumokat hoznak létre és adnak vissza
- JAVA-ban a main függvény is statikus, hogy az operációs rendszer példányosítás nélkül el tudja indítani a programot

## Statikus blokkok és inicializálás (JAVA specifikus)

- JAVA-ban létezik **statikus blokk**, ami akkor fut le amikor a JVM először betölti az osztályt
  - static {}
  - ez bonyolultabb statikus inicializálásokra való, amik nem férnek el egy sorban

## OPERÁCIÓ ÉS OPERÁTOR TÚLTÖLTÉS (OVERLOADING)

- túltöltés vagy másnéven az operáció-kiterjesztés azt jelenti, hogy ugyanazt a nevet vagy operátort több különböző célra használjuk fel

## Operáció (metódus) túltöltés (overloading)

- mindkét nyelvben (JAVA és C++) jelen van
- a **statikus polimorfizmus** egyik formája, azaz a döntés, hogy melyik változata fusson le a metódusnak már **fordítási időben** megszületik
  - lényege, hogy egy osztályon belül több metódusnak lehet ugyanaz a neve, ha a paraméterlistájuk (számosság, típus) eltérő
    - szignatúra = metódus neve, paraméterekszáma és sorrendje
      - de a visszatérési érték nem része
  - FONTOS, hogy a visszatérési érték típusa önmagában nem különbözteti meg a metódusokat egyik nyelvben sem, a fordító nem tud dönteni melyiket válassza
- leggyakoribb esetben konstruktoroknál használjuk (üres konstruktor vs paraméteres konstruktor), vagy olyan segédfüggvényeknél, mint például a max(), mely dolgozhat akár egészértékekkel, de lebegőpontos számokkal is

## Operátor túltöltés (overloading)

- C++ specifikus
  - JAVA kerüli a bonyolultság csökkentése érdekében, kivétel a + operátor Stringeknél mely be van építve, de alapvetően nincs rá lehetőség
- lehetővé teszi a bépített operátorok (pl: +, -, \*, <<, []) újraértelmezését saját típusokra
  - célja, hogy a sajátoszályainkkal ugyanolyan természetességgel végezhessünk műveleteket, mint a primitív típusokkal
  - értékadás (=) operátor nem öröklődik
  - ezeket nem lehet: ., :: (hatókör), ?: (feltételes, tenary), sizeof
- **szintaxis:** *visszateresi\_típus operátor@ (paraméterek)*
  - ahol a @ helyére az operátor kerül (pl. operator+)
- megszorítások
  - új operátorjeleket nem hozhatunk létre, csak meglévőket használhatunk
  - operátorok prioritása (műveleti sorrend) és asszociativitása (balról jobbra kötés) nem változtatható meg
  - valamint operandusok száma sem módosítható (+, az mindig 2 operandusú marad)
- típusai

- **Tagfüggvényként**

- az operátor az objektumon belül van, az első operandus mindig a this

```
class MyClass { :
public:
 int data;
 MyClass(int d) : data(d) {}
 MyClass operator+ (const MyClass& obj) {
 data += obj.data;
 return *this;
 }
};
```

- **Globális függvényként**

- gyakran friend kulcsszóval, hogy hozzáférhessen a private adattagokhoz is

- akkor szükséges ez, ha a bal oldali operandus nem a mi osztályunk

- pl: cout << saját\_objektum esetén a bal oldal egy ostream típus

```
class MyClass {
private:
 int data;
public:
 MyClass(int d) : data(d) {}
 friend MyClass operator+ (const MyClass& obj);
};
```

// A friend függvény hozzáférést kap az osztály privát adattagjaihoz

```
void operator+ (const MyClass& obj) {
 data += obj.data;
 return *this;
}
```

## KIVÉTELKEZELÉS

- lényege a hibakezelés és az üzleti logika szétválasztása
  - ahelyett, hogy minden függvény visszatérési értékét vizsgálánánk (pl: -1-e vagy null-e), inkább kivétel dobódás esetén a vezérlés egy erre felkészített szakaszra kerül
- **throw**
  - amikor hiba történik, létrehozunk egy kivétel objektumot és eldobjuk
- **catch**
  - a hívási láncba valahol egy programrész jelzi, hogy eltudja kapni és kezelni az adott típusú hibát
- **throws (csak JAVA)**
  - a metódus fejrészében jelzi, hogy a metódus milyen típusú ellenőrzött kivételt dobhat tovább

### Try-Catch-Finally (alapmechanizmus)

- **try blokk**
  - ide kerül a veszélyes kód, mely hibát okozhat
  - JAVA esetén van egy try-with-resource változata
    - lényege, hogy blokk szintű változót deklarálhatunk amelyek a végrehajtás végén megszűnnek
    - mondhatni átvette a finally szerepét
    - ```
try (FileWriter writer = new FileWriter("output.txt")) {  
    writer.write("Hello, World!");  
} catch (IOException e) {  
    System.out.println("I/O hiba: " + e.getMessage());  
}
```
- **catch blokk**
 - itt kezeljük a hibát
 - több catch blokk is állhat egymás után
 - sorrend fontos, specifikusak előbbre, általánosabbak hátra, mert általában öröklődési kapcsolatok vannak és egy sima Exception osztályt ha előbb rakunk mint egy specifikus SajatException osztály, ami az Exceptionból öröklődik akkor, nem jutunk el a SajatException catch blokkjáig, mert a sima Exception elkapja
- **finally blokk (csak JAVA)**
 - olyan kód mely mindenképp lefut, független attól, hogy történt-e hiba vagy sem
 - ide kerülnek az erőforrások (fájl, db és hálózati kapcsolatok) lezárását
 - C++-ban azért nincs, mert a RAII elv (destruktorok) végzi el az erőforrás felszabadítást automatikusan (dinamikus objektumoknál kell a delete)

JAVA hibakezelés specifikus dolgok

- JAVA-ban minden kivétel a **Throwable** osztályból származik
- kivétel-hierarchia
 - **Error**
 - súlyos rendszerhiba amiket nem kell / nem lehet kezelni
 - pl: OutOfMemory error
 - **Exception**
 - a program által kezelhető hibák
 - két fő csoportja van:
 - **Checked (Ellenőrzött) kivételek**
 - a fordító kényszeríti a kezelésüket, tehát vagy try-catch vagy továbbdobás a metódus szignatúrájában a throws kulcsszóval
 - pl: IOException
 - **Unchecked (Nem ellenőrzött) kivételek**
 - a RuntimeException leszármazottai
 - nem kötelező kezelni őket, általában programozói hibák
 - pl: NullPointerException, ArrayIndexOutOfBoundsException

C++ hibakezelés specifikus dolgok

- C++-ban bármit el lehet dobni, akár például int-et, de illik az std::exception osztályból származtatni amit eldobunk
- **Stack Unwinding (verem-visszacsévelés)**
 - amikor kivétel keletkezik, a vezérlés kilép az aktuális függvényből, és elindul felfelé a hívási láncon, keresve egy megfelelő catch blokkot, eközben a rendszer automatikusan meghívja a lokális objektumok destruktoraikat azokon a szinteken, amiket elhagy
 - ez garantálja, hogy hiba esetén se legyen memóriaszivárgás (ez a RAII - Resource Acquisition Is Initialization elv alapja)
- catch(...), azaz a három pont segítségével minden típusú kivételt elkap (univerzális elkapó)
- C++ destruktorból soha ne engedjük ki kivételt, mert ha már zajlik egy kivételkezelés (stack unwinding) akkor a program azonnal leáll (std::terminate)